

DSim: A Distributed Mobsim, Ready to Use

Janek Laudan , Simunto GmbH, laudan@simunto.com
Christian Rakow , Simunto GmbH, rakow@simunto.com

Motivation

At last year's Multi-Agent Transport Simulation (MATSim) User Meeting, we introduced DSim, a message-passing based mobility simulation (mobsim) implementation (Laudan et al., 2025a). Based on an earlier prototype written in the Rust programming language (Laudan et al., 2025b), the presented mobsim implementation partitions the simulated network through domain decomposition and distributes the simulation work onto isolated processes. These processes exchange simulation state across partition boundaries, using explicit messages. Three design goals guided the implementation: it must run inside the JVM, it must stay compatible with the existing MATSim ecosystem, and it must work as a drop-in replacement for QSim, the current standard mobsim implementation.

This year, we report on one year of progress for the DSim implementation. Since the last presentation, the code has been moved into the core MATSim repository and is publicly available under the GPL 2.0 license. Being part of the MATSim core makes it possible to use DSim with minimal overhead by simply adjusting the main configuration file. Additionally, substantial work has gone into further performance improvements to take advantage of DSim's distributed architecture. Lastly, we have introduced new APIs to facilitate extensions. This abstract presents the progress from an architectural prototype into a usable, documented feature of MATSim.

Using DSim as mobsim

Switching to DSim can be done entirely at the configuration file level. The simplest way to enable it is to configure it in the controller module of the corresponding `config.xml` file.

```
<module name="controller">
  <param name="mobsim" value="dsim" />
</module>
```

For more control, a new DSim config module has been added, which allows setting parameters such as the simulated network modes, the partitioning algorithm, or the number of threads, as shown below. The configuration module provides more parameters, which can be obtained from the documentation¹.

```
<module name="dsim">
  <param name="networkModes" value="car,bike" />
  <param name="partitioning" value="metis" />
  <param name="threads" value="48" />
</module>
```

The domain decomposition concept introduces substantial changes to the state management of the simulation. The default mobsim implementation, QSim, provides APIs to access the global simulation state. For example, it is allowed to query any agent in the simulation and also alter its state at any given time. This is no longer possible in DSim, as each simulation process is only aware of the agents it currently manages on its network partition. Once agents cross a domain boundary, only the new simulation process has access to their state; the simulation process they left can no longer query it. Extensions which query and alter agent state during the simulation must keep track

¹<https://github.com/matsim-org/matsim-lib/blob/main/contribs/distributed-simulation/README.md>

of which agent is managed by which process. DSim provides two main interfaces to achieve this: `NotifyAgentPartitionTransfer`, and `NotifyVehiclePartitionTransfer`. Extensions may implement these interfaces, so that they are notified by a simulation process when agents and vehicles cross domain boundaries. The notification includes the agent/vehicle as well as the target partition. With this information, the corresponding extension can update its internal bookkeeping and transfer state to the corresponding partition.

On the other hand, extensions managing the movement of agents and vehicles must be aware of the partitioned simulation domain and manage the transfer of agents and vehicles once they cross domain boundaries. Such extensions are also obliged to notify the simulation process that vehicles and agents have left or entered a partition, so that the notification mechanism described above works.

In addition to new APIs to track domain boundary crossings, the programming interface for exchanging messages has been refined. Each simulation process provides an injectable `PartitionTransfer` object, which extensions can use to transfer information between partitions in the form of messages. Messages are collected during one simulation step and are sent to the corresponding partitions at the end of the simulation step, minimizing messaging overhead. On the receiver partition, messages are dispatched to implementors of `DSimComponentsMessageProcessor`. Implementors of this interface specify which messages they are interested in, so that each implementor only receives messages relevant to them.

Performance Improvements

Despite its domain decomposition architecture, the initial DSim version committed to the main repository shows limitations in scalability for higher process counts. Extensive profiling shows that this problem is related to the backward compatibility of DSim, in particular to the event handling architecture of MATSim. By default, one global event handler is registered for a certain task. This does not play well with the distributed architecture of DSim, as all events produced by parallel simulation processes must be dispatched to a single event-handling process.

DSim introduces the `DistributedEventHandler` annotation with which event handlers can signal how events should be processed. For instance, it is possible to indicate that an event handler should be executed directly on the simulation process so that this computation becomes distributed in the same way the simulation is. As part of the performance optimization work, compute-heavy event handlers are updated to be computed in a distributed fashion. In particular, switching the event handling of the scoring module proves to be beneficial, as it consumes almost all events produced during the mobsim.

Figure 1 shows the real-time ratio (RTR) of a preliminary benchmark comparing the QSim and DSim implementations for a 25% sample of Switzerland's population (2.5 million agents). The benchmark is executed on a machine with two AMD EPYC 9654 CPUs featuring 96 cores each and a maximum clock rate of 5.7 GHz. Figure 1 shows that DSim consistently outperforms the QSim implementation even on a single process. The RTR increases up to 64 cores, reaching a maximum of 387.9 at 96 cores. The QSim implementation also reaches its maximum RTR of 74 for 96 cores. The benchmark indicates that DSim can execute the same simulation scenario 5.5 times faster than the current default mobsim implementation.

Conclusion

Over the past year, DSim has matured from an architectural prototype into a usable feature of MATSim. Its integration into the core MATSim repository, together with the new configuration module and extension APIs, allows users to adopt DSim as a drop-in replacement for QSim with

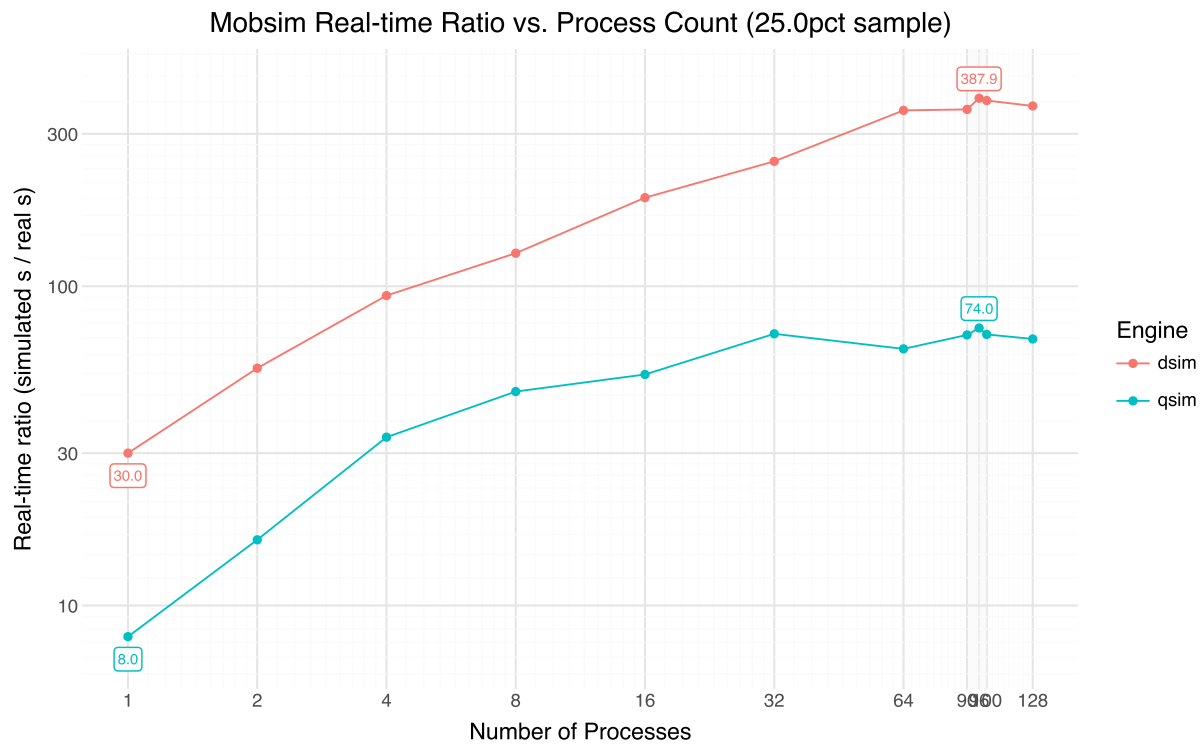


Figure 1: Real-time ratio for computing a benchmark Switzerland Scenario with a sample size of 25% (2.5 million agents) with the QSim and with the DSim implementation

minimal setup effort. At the same time, targeted performance improvements, in particular the distributed handling of events, have substantially increased DSim’s scalability. The presented benchmark demonstrates that DSim can execute a large-scale scenario up to 5.5 times faster than QSim, confirming that the distributed architecture delivers on its performance promise while remaining compatible with the existing MATSim ecosystem.

References

- Laudan, J., Heinrich, P., Nagel, K., 2025b. High-performance mobility simulation: Implementation of a parallel distributed message-passing algorithm for MATSim. *Information (Basel)* 16, 116.. <https://doi.org/10.3390/info16020116>
- Laudan, J., Rakow, C., Rieser, M., Axer, S., Rewald, H., 2025a. DSim – A distributed message-passing mobsim implementation [WWW Document].. URL https://matsim.org/conferences/mum2025/abstracts/MUM25_paper_18.pdf